

Haskell Style Guide

The syntax of the Haskell language can be really beautiful. Even if you don't continue to write programs for GHC after the end of the semester, you may continue to use this syntax to express your ideas, because it just works. Furthermore, many people will be reading the code that you produce in this course—you, your partner, the course staff, and perhaps the whole class during a homework discussion.

So make it pretty! The style guidelines below help. But the best advice is to revise, revise, revise...

Some of these style guidelines can be checked automatically by the `hlint` tool; feel free to run it on your assignment before you submit. You can install this tool by running the following command (you might need to install “`libncurses-dev`” on WSL):

```
stack install hlint
```

NOTE: `hlint` may give you suggestions for Haskell features and functions that we may not have yet covered in class. If an `hlint` suggestion doesn't make sense to you, and is not listed below, you can ignore it.

Required Elements

RD: Every top-level function must have an explicit type declaration

For example, write

```
myfunc :: Integer -> Integer -> [Integer] -> [Integer]
myfunc x y z = (x+y) : z
```

instead of just:

```
myfunc x y z = (x+y) : z
```

These declarations will sometimes be fairly obvious (from the function definition itself, its name, nearby comments, or whatever), but getting into the habit of writing them all the time is good discipline. And, good type signatures lead to better error messages.

RT: Every significant piece of functionality must have testing code

In particular, every solution to a homework exercise must include a “test driver” that shows how it behaves on several examples. These test cases will often be given to you. If that is the case, you do not need to add more.

RC: Submitted code must compile (cleanly)

Any code you submit must be accepted by GHC without errors or warnings. Never submit anything that you have changed, no matter how small the change, without checking it with GHC.

Formatting

FT: No tab characters

Do not use the tab character (0x09). Instead, use spaces to control indenting. This is because the width of a tab is not uniform across all computers, and what looks good on your machine may look terrible on ours, especially if you have mixed spaces and tabs. The homework assignments include a flag that warns against inadvertent tab insertion. Don't remove this flag.

F8: 80-column lines

No line in your program should be longer than 80 characters. Although most screens are wide enough now to display much longer lines, there are still many editors whose default width is 80 characters, and 80-character lines are the natural width for printing programs in reasonable-sized fonts.

FO: Whitespace

Use whitespace to delimit your code and make it easier for humans to parse. Surround binary operators with a single space on either side. Put a blank line between top-level definitions, though no blank lines between type signatures and function definitions. Align the elements in declarations and expressions that span multiple lines.

Naming

ND: Use descriptive names

Choose names that reflect the intended use of the value referred to by the name. Names are documentation. As a general rule, short names (one or a few characters) are appropriate for variables with small scopes, like local definitions or parameters to functions, whereas longer names are appropriate for global definitions, such as top-level functions.

NC: Follow standard Haskell naming conventions

- For long names composed of multiple words, use “camelCase.” Smash the words into a long string and capitalize the first letters of the second and following words— e.g., `veryLongFunctionName` instead of `very_long_function_name` or `verylongfunctionname`.
- Module names should begin with capital letters.
- When pattern matching against a list, if you call the head `x`, then use `xs` for the tail.
- For better readability, don't capitalize all letters when using an abbreviation. For example, write `HttpServer` instead of `HTTPServer`. Exception: Two-letter abbreviations, such as `IO`.

Comments

C: Use comments

Most top-level declarations (data types and functions) should be preceded by a short comment, written with Haddock syntax. In a few cases, this will not be necessary (see CO).

```
-- | sums a list of integers
sum :: [ Integer ] -> Integer
sum = foldl (+) 0
```

CW: Comments should say what the code does, not how

It should be obvious how the code works, just by looking at it. If this is not the case, you need to rewrite the code.

CO: Do not over-comment

Very many or very long comments (especially within the body of a function) are more distracting than helpful. Long comments may appear at the top of a file or section of code, if you need to explain the overall design of the code or refer to sources that have more information about the algorithms or data structures. All other comments in the file should be as short as possible. Judicious choice of variable names can help minimize the need for comments.

Avoid comments that state the obvious:

```
-- | This function increments its argument
inc :: Integer -> Integer
inc x = x+1
```

CE: Use proper English

Comments need not always be written in complete sentences, but when they are, standard rules of English grammar apply. Spelling also counts.

Functionality

FS: Write simple functions

“Functions should be short and sweet, and should do just one thing. They should fit on one or two screenfuls of text (the ISO/ANSI screen size is 80x24) and do one thing and do that well.” From the Linux kernel coding style

FD: No dead code

Do not leave unused or commented out code in your submission. It detracts from the story line of your program. If you are worried about saving this code for later, copy it to a different file or use version control.

PF: Avoid partial functions

Your code should always return an answer, not error. Some prelude and standard library functions are partial. Avoid these in your code! In particular, never use: head, tail, or fromJust in this class. Challenge yourself to find a better way.

Pattern Matching

PI: No incomplete cases

Incomplete pattern matches are flagged with compiler warnings, which are treated as errors for grading purposes.

PF: Match in the function arguments

Tuples, records and datatypes can be deconstructed using pattern matching. If you simply deconstruct the function argument before you do anything useful, it is better to pattern match in the function argument. Consider these examples:

Bad

```
f arg1 arg2 = ... where
  x = fst arg1
  y = snd arg1
  z = fst arg2
```

Good

```
f (x,y) (z,_) = ...
```

PN: Combine nested cases

Rather than nest case expressions, you can combine them by pattern matching against a tuple, provided the tests in the case expressions are independent. Here is an example:

```
case x of
  Red -> case y of
    Red  -> True
    Blue -> False
  Blue -> case y of
    Red  -> False
    Blue -> True
```

vs.

```
case (x,y) of
  (Red,  Red) -> True
  (Blue, Blue) -> True
  (  _,   _) -> False
```

Verbosity

VL: Don't rewrite library functions

The Haskell library has a great number of functions and data structures – use them! Often students will recode filter, map, and similar functions. Hoogle can help you find them.

Exception: Sometimes the homework may ask you to rewrite certain library functions for practice.

VI: Misusing if

Remember that the type of the condition in an if expression is `Bool`. If the result type of the if expression is also `Bool`, then you probably should not be using if at all. Consider replacing the expression on the left with the one on the right.

Verbose expression	Cleaner expression
if e then True else False	e
if e then False else True	not e
if e then e else False	e
if not e then x else y	if e then y else x
if x then True else y	x y
if x then y else False	x && y
if x then False else y	not x && y
if x then y else True	not x y

Site proudly generated by [Hakyll](#)